CAS R&D AI LAB • IMPLEMENTATION SERIES

HELM-Q1

Quantum-Assisted CAS Execution

An executable build directive for connecting RPA, RPM, CJTs, Noetics, and HELM to cloud quantum processors through independently verified classical control.

> **IMPLEMENTATION WORK ORDER**
> Local simulation first. Free hardware access next. Paid experiments only after explicit budget admission.

Architecture
Classical HELM host + bounded quantum worker.

First milestone
Construct, verify, retain, restart, and reuse one procedure.

Provider decision
IBM Open for initial hardware access; Amazon Braket for low-dollar comparison.

Prepared for Little Henderson

Version 1.0 • September 27, 2026

Contains cloud research, staged work orders, cost controls, and acceptance criteria.

Contents

Executive decision 3
1. Source custody and existing work 3
2. Authority contract 4
3. Target execution path 4
4. Component implementation map 5
5. Phase Q0: inventory, baseline, and host sizing 5
6. Phase Q1: typed contracts and isolation 6
7. Phase Q2: classical reference problem and verifier 6
8. Phase Q3: reversible compiler and local quantum search 7
9. Phase Q4: IBM and Amazon Braket hardware adapters 8
10. Phase Q5: RPM/RPA integration and retained reuse 8
11. Phase Q6: restart-safe jobs, cost protection, and deployment 9
12. Phase Q7: comparative evaluation and claim discipline 9
13. Follow-on track A: quantum RPM backtracking 10
14. Follow-on track B: constrained optimization 10
15. Follow-on track C: probabilistic scenario estimation 10
16. Follow-on track D: quantum CAS profile and native boards 11
17. Work allocation and handoff 11
18. Proposed implementation tree and commands 11
19. Minimum acceptance matrix 12
20. Cloud decision record 13
21. Cloud runbook 14
22. Required final delivery from the implementation team 15
Sources 16

Read this distinction first. The 40 acceptance items are requirements for the implementation team, not completed HELM tests. The accompanying offline helper tools are implemented and tested separately. No quantum-provider calls or HELM deployment were performed.

Executive decision

Build a **classical HELM/CNCM host with a separately bounded quantum research worker**. HELM's model, memory, scheduler, RPM continuations, verifier, and commitment authority remain on conventional compute. A quantum processor receives a compiled circuit or optimization problem, not an unchanged HELM repository or model checkpoint. Amazon Braket explicitly supports this hybrid execution pattern, including custom containers for the classical algorithm. [S1, S2]

Use three implementation targets in order: a local simulator; IBM Quantum Open Plan for a small real-hardware experiment; Amazon Braket pay-per-task hardware for a separately approved low-dollar comparison. Provider adapters must be interchangeable at the typed problem/result boundary, not assumed identical at the native circuit boundary. D-Wave optimization, quantum estimation, and quantum-native CAS boards are later research tracks.

First result to earn: A bounded missing-procedure request enters through a snapshot-bound CAS request, a compiled quantum search proposes a candidate, an independent classical checker accepts it on its stated finite domain, the existing authorized retention path stores it in research memory, and a fresh process reuses it without another quantum call.

This result would establish a working quantum-assisted construction path. It would not establish broad learning, native HELM self-improvement, quantum advantage, consciousness, or a fully quantum reasoning machine.

1. Source custody and existing work

The private repository `LittleHenderson/noetic-compass` was reachable during this preparation. The current coordination artifact read was `OWNER_CORRECTION_ROUND.txt` on `codex/helm-three-lead-delivery-20260927`, Git blob `dfd0b65104b88052419213627620a926d9838d42`. It preserves active HELM replication, existing ownership, protected evidence, historical failed attempts, and research-only retention. [P1]

That read was not a full runtime inspection. No current HELM RAM/VRAM profile, checkpoint compatibility, deployment readiness, remaining cloud allowance, or complete test baseline was verified here. An earlier runtime-path lookup on the default branch failed; do not interpret that as proof that HELM does not exist. Treat source reconciliation as the first engineering gate.

The quantum work must not overwrite active HELM worktrees, rename existing CAS semantics, replace the transaction coordinator, change current training, or reuse a stale known-failing cloud freeze. Preserve compiler-assisted versus learned behavior as separate attributions. A hash proves byte identity, not permission; admitted parent identity must come from the trusted host, not agreement among caller-supplied records. [P1]

All paths and interfaces proposed below are **new work-order targets**, not claims that these modules already exist. Codex must map them to the admitted source tree before implementation.

2. Authority contract

Component	Allowed responsibility	Authority it must not acquire
HELM/CNCM host	Form a bounded problem from admitted state; choose allowed execution mode	Bypass source, budget, or acceptance rules
RPA adapter	Read snapshot-bound states and typed relations; display results	Turn screen focus or graph navigation into execution
RPM adapter	Preserve caller, branch, immediate parent, continuation, and prerequisite state	Declare a parent complete merely because a descendant returned
Quantum compiler	Translate an approved finite problem into a circuit/model	Read hidden answers; rewrite the acceptance criteria
Quantum worker	Submit allowed jobs and decode candidate samples	Write authoritative model, memory, source, or checkpoint state
Independent verifier	Check the candidate against the actual problem and admitted snapshot	Treat measurement frequency or low energy as proof of correctness
Existing host coordinator	Apply the existing authorized research-retention or commitment path	Delegate authority to the quantum provider or viewer
Budget controller	Admit or refuse requested resource use	Change its own spending authority

Provider credentials must exist only in the worker's execution environment. Keep QPU submissions free of secrets, protected evaluation data, private customer content, or unrelated repository history. Begin with synthetic fixtures.

3. Target execution path

`HELM request -> admitted state snapshot -> bounded RPA/RPM problem -> classical eligibility checks -> quantum compiler -> resource and budget admission -> provider job -> decoded candidates -> independent verification -> existing research-retention path -> fresh-process reuse`

The normal classical path must remain available. Support `disabled`, `shadow`, and `research` modes. Disabled mode must preserve the admitted baseline. Shadow mode may compare results but cannot change the authoritative answer. Research mode may retain accepted research artifacts only through the existing host authority contract.

Separate the persistent HELM application from disposable experiment workers. No interactive request should wait indefinitely for a cloud queue. A deadline produces a typed timeout or classical fallback, not an unbounded retry loop.

4. Component implementation map

CAS item	Required implementation work	Acceptance boundary
RPA	Export a finite, typed search space with stable snapshot-bound identifiers	Geometry is a view; containment, refinement, and reachability stay distinct
RPM	Export bounded prerequisites and exact return identities	Cycles, shared prerequisites, and depth limits are explicit; return to the immediate caller
CJT	Define finite encodings for approved Cause/Bias/Effect transformations	Proposed effects and bias updates do not become authoritative inside the circuit
Noetics	Inventory operations and classify reversible, reversibly embeddable, measurement-based, or unsupported	No unsupported operator is silently replaced with a convenient quantum gate
D/W/P and Foundations	Convert only documented measurable criteria into predicates or objectives	Semantic names are not automatically numerical observables
CAS-Ohm / ASOM	Start as classical eligibility, diagnostic, and routing signals	Neither replaces quantum noise characterization or error correction
Compiler	Add a typed Quantum Problem IR and separate backend lowering	Preserve bindings, ordered composition, types, operation versions, and domain
Memory	Store verified classical artifacts, provider receipts, and reuse provenance	Never claim to snapshot arbitrary unknown coherent quantum state
HELM self-improvement	Later route proposed modifications through the existing isolated test/reload process	Externally authored quantum/compiler assistance is not native HELM invention

The RPA/RPM/CJT mappings above are proposed extensions. Source-bound semantic verification is mandatory before calling a fixture CAS-native.

5. Phase Q0: inventory, baseline, and host sizing

Responsible lead: Codex, with source-semantic review by Claude and independent evidence review by DeepSeek where those existing leads are available.

Read the applicable ownership, authorization, source inventory, operation contracts, current job ledger, and protected-test rules. Resolve the actual HELM executable entry point, runtime languages, dependency locks, compiler interface, model/checkpoint requirements, RPM continuation type, and commit/retention entry point.

Create `SOURCE_INVENTORY.json` containing exact admitted source references, relevant file hashes, command lines, checkpoint digest or legitimate null, environment identity, and supported execution modes. Do not make a laptop run a prerequisite when that environment is not authorized. [P1]

In an authorized environment, measure cold-start time, peak resident memory, checkpoint disk footprint, CPU throughput, and, where applicable, accelerator type and peak VRAM. Measure inference and training separately. Record whether Linux x86-64 and Linux Arm64 are actually supported. A proposed 4 GB server is a sizing hypothesis, not a verified HELM requirement.

Reproduce the relevant baseline with the quantum feature disabled. Store collected test identifiers separately from executed results; preserve failures and exact commands. Capture deterministic state parity separately from timing and wall-clock receipt identity.

Exit Q0: An admitted source manifest, reproducible baseline, profile report, ownership acceptance, and a compatible deployment target. Missing source or authorization produces a precise blocker, not a guessed replacement.

6. Phase Q1: typed contracts and isolation

Implement a new `QuantumProblem` record with: schema/version; request ID; admitted parent reference; source/compiler/operation digests; state snapshot reference; branch ID; caller frame; return continuation; problem family; finite input domain; instruction library; maximum program length; prerequisites; goal predicate; decoding order; resource limits; deadline; mode; and independent-verifier reference.

Implement `QuantumProposal`, `QuantumRunReceipt`, and `QuantumVerification` records. The receipt must distinguish provider-reported hardware execution from local or cloud simulation. Include requested and actual backend identity, device properties snapshot, native circuit digest, compilation settings, measurement order, shots, observed counts, job IDs, timings, quote, estimated charges, and reconciled charges when available.

Validate every deserialization route, not only a friendly constructor. Reject unknown schemas, stale parent state, wrong branch, malformed checkpoint, unsupported operations, and missing resource bounds. An informational JSON authorization field is not itself a trusted grant.

Create a backend interface with `capabilities`, `compile`, `estimate`, `submit`, `poll`, `cancel`, and `decode`. A request that requires an unsupported capability fails explicitly. No provider adapter has a direct reference to a writable host state store.

Exit Q1: Contract tests, adversarial input tests, disabled/shadow parity, and a demonstrated refusal of unauthorized state writes and submissions.

7. Phase Q2: classical reference problem and verifier

Start with a fully specified instruction-synthesis fixture. Use a two-bit value `x` in `{0,1,2,3}` and four public instructions:

Code	Operation	Exact semantics
00	ID	x
01	INC	(x + 1) mod 4
10	DEC	(x + 3) mod 4
11	NOT2	3 - x

Search ordered programs of length three. There are `4^3 = 64` encodings. The positive task is to implement `(x + 2) mod 4` for **all four inputs**, not just one example. Add a negative task requesting `x mod 2`; it cannot be realized by compositions of these bijections. This negative example has a direct mathematical impossibility proof and a finite exhaustive reference check.

This is an algorithm-development fixture, not yet a verified mapping of actual HELM Noetics. After source reconciliation, replace or extend the operations with the smallest admitted CAS subset whose semantics are independently checkable.

Implement exhaustive search, uniform random candidate sampling, and structured/pruned classical search. Add a constraint-programming or SAT baseline when the representation supports one. Share public task specifications, never private answer lists.

The independent verifier receives a candidate instruction sequence and recomputes behavior directly. It must not delegate its judgment to the quantum oracle or trust a returned "valid" flag. A future larger-domain fixture must state whether verification is exhaustive, theorem-backed, or test-only.

Exit Q2: Correct positive and no-solution fixtures, independent checks, reproducible baseline measurements, and separate public-spec/evaluator storage.

8. Phase Q3: reversible compiler and local quantum search

Compile the approved predicate into a reversible computation. For a finite classical function, an embedding of the form `|input,z> -> |input,z XOR f(input)>` can retain inputs while writing output workspace. The implementation must account for workspace, overflow conventions, invalid encodings, and inverse cleanup rather than simply relabeling an irreversible operation.

For the initial fixture, use six program-selection qubits plus separately counted data, checker, and temporary registers. Do not describe six qubits as the complete machine. The actual total is a compiler output, not a number assumed in advance.

Build the predicate by conditionally executing the selected instructions, checking each public input/target condition, applying the phase mark, and uncomputing temporary state. Preserve the selection register. Test every selection basis state, coherent superpositions, register ordering, and ancilla cleanup.

Forbidden shortcut: solving the problem classically, encoding the successful program IDs in a lookup oracle, and presenting its output as quantum procedure discovery. A lookup-oracle circuit may be a labeled transport/calibration test, but it cannot pass the constructive-oracle or advantage gate.

Use amplitude amplification as the initial algorithm. Handle unknown solution count with a justified bounded schedule or separately accounted counting procedure. Do not set the number of iterations using hidden evaluator answers. No returned candidate after a finite sample budget means `NO_CANDIDATE_WITHIN_BUDGET`, not `UNSAT_PROVED`.

Verify on an ideal simulator, then a documented noise model. Record all preparation, compilation, oracle, measurement, and verification costs. A simulator run establishes simulation behavior; it does not establish a hardware speedup. [S11]

Exit Q3: Predicate equivalence across the small complete domain, clean workspace, cross-implementation decoding agreement, simulator receipt, and no answer leakage.

9. Phase Q4: IBM and Amazon Braket hardware adapters

Implement IBM first behind an explicitly selected Open Plan instance. Use a current supported Qiskit Runtime job API and compile to the chosen device's instruction set. Record exact installed dependency versions in a lock file. Do not copy obsolete runtime examples without checking current documentation. Open Plan does not permit sessions; use a supported execution mode. [S3, S12]

Then implement a Braket adapter using a current supported gate-based device. Query its current properties and status before compiling or submitting. The September 27 device catalog includes Rigetti Cepheus-1-108Q and IQM Garnet/Emerald; a listing does not prove account access or immediate availability. QuEra Aquila uses analog Hamiltonian simulation and is not a drop-in target for the same gate circuit. [S4]

Keep native circuit lowering separate for each backend. Test measurement endianness, qubit layout, unsupported gates, task-size limits, shot bounds, queue timeout, and partial failure. Do not assume an OpenQASM export transfers every backend feature.

First execute a minimal hardware transport check and label it as such. Only then submit a resource-admitted constructive search fixture. Hardware admission requires a fresh quote, authenticated allowed plan/device, remaining quota or approved budget, an independent compiler check, and no existing job ownership conflict.

Exit Q4: At least one independently validated candidate from an actual QPU, complete hardware provenance, and a separate result for every failed or refused attempt. Simulator success cannot stand in for this gate.

10. Phase Q5: RPM/RPA integration and retained reuse

Connect the quantum worker as a candidate-producing subroutine, not a second reasoning coordinator. Export the actual snapshot, caller, frame, branch, and continuation identity. Keep inspection focus distinct from the current executable state.

Introduce two nested prerequisites, then a shared prerequisite. Test the immediate-parent return path, remaining work, isolation between speculative branches, rollback, and cycle/depth refusal. A graph with shared prerequisites is not automatically covered by a tree-search theorem. [S13]

Pass the sampled candidate through the independent verifier and existing retention authority. Save only the classical verified procedure and relevant provenance. Restart a fresh process, disable construction and quantum calls, and show that the retained procedure still solves the public task.

Repeat checkpoint/reload failure injections around request creation, admission, provider submission, result persistence, verification, and retention. Never attempt to serialize an arbitrary live coherent quantum state as a normal RPM checkpoint.

The viewer must separately label declared search space, actual observed samples, and verified retained outcome. It must not depict every theoretical branch as an observed hardware execution.

Exit Q5: A source-bound request-to-verified-retention-to-reuse episode, exact caller resumption, and preserved baseline behavior.

11. Phase Q6: restart-safe jobs, cost protection, and deployment

Persist a job intent and reserve its maximum planned cost before submission. Store a stable idempotency key, problem/circuit hashes, provider, allowed device, shot count, and deadline. Reuse the provider's idempotency mechanism where supported. Otherwise, reconcile an uncertain submission before retrying. A lost network reply must not automatically create a second paid job.

Use a state machine such as `PREPARED -> ADMITTED -> SUBMITTING -> SUBMITTED -> RUNNING -> RESULT_STORED -> VERIFIED -> RETAINED`, with explicit refused, cancelled, failed, uncertain, and timed-out alternatives. Provider details may differ. Never treat application timeout as proof that a billable task was cancelled.

Require atomic accounting of spent plus reserved in-flight costs. The worker cannot increase its budget, substitute an expensive device, enable a paid fallback, or delete provider-side caps. Keep estimated charges distinct from provider-reconciled charges.

Braket now documents device-level spending limits that consider current and queued spend. Create the limit before the pilot's first task. They do not cover simulator, notebook, classical Hybrid Jobs, or reservation charges, and already-created tasks are not retroactively protected by a new limit. Use separate controls for those resources. [S5]

Containerize only after Q0 confirms the runtime and platform. Keep the admitted checkpoint read-only, use a separate persistent research-data directory, restrict the network and credentials, and run as a non-root user. Preserve artifact digests and dependency locks. No guessed HELM server command should appear in the deployment image.

Exit Q6: Restart-safe handling, duplicate-charge prevention tests, cost reconciliation, credential isolation, shutdown verification, and an approved host resource envelope.

12. Phase Q7: comparative evaluation and claim discipline

Freeze problem families, public specifications, held-back evaluation custody, seeds, budgets, success criteria, and stopping rules before a paid comparison. Use the same task semantics and acceptance criteria across classical, simulated quantum, and actual hardware runs.

Measure total wall time and separate compilation, state preparation, queue time, device execution, decoding, verification, and persistence. Report memory, logical/physical qubit needs where meaningful, ancillas, circuit depth, two-qubit gates, oracle uses, samples, success rate, repeated-run uncertainty, billed resource use, and total cost to a verified solution.

Retain failure rates and no-solution cases. Preprocessing and classical oracle construction are part of the computation. Compare against a strong classical strategy, not only exhaustive enumeration. Tiny fixtures are integration tests, not a credible demonstration of advantage.

Use distinct labels: `CONTRACT_VALIDATED`, `SIMULATOR_VALIDATED`, `QPU_EXECUTED`, `CAS_EPISODE_VALIDATED`, `RETAINED_RESEARCH_ONLY`, and `COMPARATIVE_RESULT`. Do not make one status imply the next. A claim of advantage must name its task family, baseline, precision/success requirement, resource metric, and tested size range. [S13, S14]

Exit Q7: An independently reviewed report that may conclude positive, negative, or inconclusive comparative value. A negative result does not erase the successful engineering work.

13. Follow-on track A: quantum RPM backtracking

After Q5, formalize the partial-assignment state, root, child generation, rejection predicate, completion predicate, search bounds, and maximum depth. Characterize the actual reachable tree or graph and the cost of coherent access to its rules.

Implement a quantum walk/backtracking experiment only after deriving which theorem assumptions the representation satisfies. Montanaro's backtracking result provides bounded-error speedups under specific access assumptions and includes factors beyond the square root of tree size. It is not a turnkey theorem for an arbitrary mutable interactive runtime. [S13]

Shared dependencies, changing knowledge, external tool calls, and non-reversible host state require explicit treatment. Freeze a subproblem before coherent execution and return a classical candidate when complete.

Deliverables: mathematical reduction, oracle implementation, resource estimate, simulator comparison, and a hardware feasibility decision. Do not require a deep circuit to run on a device that cannot meaningfully execute it.

14. Follow-on track B: constrained optimization

Build a separate QUBO/Ising or constrained-model adapter for board/resource/prerequisite allocation. Define binary decisions, feasible schedules, a documented objective, and exact post-sample hard checks. Set penalties from proven bounds or validate them exhaustively on small cases; do not choose arbitrary large weights and assume constraints are enforced.

Compare classical exact solving and heuristics with a D-Wave annealing/hybrid solver or gate-based QAOA where appropriate. Record whether the provider performed annealing or a proprietary hybrid computation. Do not attribute all hybrid performance to the QPU without evidence. D-Wave Leap documents annealing systems and hybrid optimization services; its advertised trial is not an assumed ongoing free allocation. [S8]

Deliverables: model equivalence tests, feasibility checks, baseline suite, cost estimate, and a separately approved provider experiment. No low-energy sample can bypass the host acceptance gates.

15. Follow-on track C: probabilistic scenario estimation

Specify a generative scenario computation and bounded numerical payoff. Identify how coherent preparation, inverse preparation, and the payoff marking would be implemented. Compare amplitude-estimation variants with classical Monte Carlo at matched error probability and precision.

Count state preparation, oracle compilation, repeated circuits, noise, and full statistical error. Quantum Monte Carlo speedups apply under explicit access and variance assumptions; faster estimates of a model do not make the model more truthful about the world. [S15]

Deliverables: exact small-instance ground truth, error/variance contract, preparation circuit, reproducible comparisons, and a decision whether the resource demand justifies hardware use.

16. Follow-on track D: quantum CAS profile and native boards

For a normalized 40-component descriptor, test the proposed amplitude encoding `sum_i sqrt(p_i) exp(i*phi_i) |i>`, with unused basis states explicitly defined. Six qubits provide 64 basis labels, enough labels for this profile but not a complete reasoning machine.

Start with zero phases and recover the expected measurement distribution. Then preregister a meaning and operational purpose for any phase parameters. Test normalization, zero entries, unused-state leakage, state preparation cost, and the measurements actually needed. A whole-distribution reconstruction can be expensive; avoid reading back every amplitude when the task needs one statistic.

For native CJT/Noetic boards, first classify each admitted operator's finite semantics and give its unitary extension, dilation, or measurement semantics. Verify composition, inversion where defined, and cleanup. Do not assume every CAS transformation is unitary. Compare a small parameterized board against matched classical alternatives before attaching it to HELM learning.

Deliverables: source-semantic mappings, preparation/measurement contracts, resource estimates, matched baselines, and research-only integration proposals. This track does not authorize neural expansion, model retraining, or self-modifying code.

17. Work allocation and handoff

Preserve the existing three-lead ownership arrangement rather than launching a new agent swarm. The following scopes are proposed assignments, not evidence that any agent has accepted them. [P1]

Lead	Proposed scope	Required handoff
Codex	Source/runtime inventory; typed worker integration; provider adapters; containers; job lifecycle; resource ledger	Exact commit, commands, environment, test receipts, remaining blockers
Claude	CAS/RPM/RPA semantics; admitted operation mappings; reversible compiler review; return-address correctness	Source-bound semantic decisions, counterexamples, corrected cases
DeepSeek	Independent oracle-leakage checks; adversarial authority tests; resource/cost audit; benchmark challenge	Independently executed evidence or explicit unexecuted review findings

Do not create additional workers merely to reconcile bookkeeping. Reuse accepted worker limits and isolated worktrees. Record actual owner acceptance and accessible transport paths. A reviewer's authored fix must retain that attribution and receive a distinct check where needed. No agent may self-award a passing acceptance gate.

18. Proposed implementation tree and commands

Map this proposed tree into the admitted repository; do not infer that it exists today.

```
research/helm_quantum/
  README.md
  SOURCE_INVENTORY.json
  config/policy.json
  schemas/{problem,proposal,receipt,verification}.json
  contracts.py
  eligibility.py
  compiler/{ir,reversible,ibm_lowering,braket_lowering}.py
  backends/{base,local,ibm_open,aws_braket}.py
  verifier.py
  job_store.py
  budget.py
  adapters/{helm,rpm,rpa}.py
  fixtures/public/
  tests/
  deploy/
  evidence/
```

The implementation must expose commands equivalent to the following **new target interface**. These commands are requirements for Codex to implement, not already installed software:

```
python -m helm_quantum inventory --read-only
python -m helm_quantum baseline --mode disabled
python -m helm_quantum validate --fixture q1-mod4
python -m helm_quantum compile --fixture q1-mod4 --backend local
python -m helm_quantum compare --fixture q1-mod4 --offline
python -m helm_quantum estimate --request REQUEST.json
python -m helm_quantum submit --request REQUEST.json --authorization TRUSTED_GRANT
python -m helm_quantum reconcile --job JOB_ID
python -m helm_quantum verify --receipt RECEIPT.json
python -m helm_quantum replay --episode EPISODE_ID --quantum-disabled
```

The delivered package contains separate, **actually runnable offline helper tools** under `tools/`. They calculate illustrative QPU quotes and validate a planning policy. They do not implement these quantum commands or authorize cloud use.

19. Minimum acceptance matrix

The following are required test IDs, not reported passes of HELM. Expand them where an existing CAS contract is stronger.

IDs	Required coverage
T01-T05	Admitted source/parent identity; unknown schema; malformed checkpoint; foreign branch; stale snapshot
T06-T10	Ordered instruction semantics; all finite inputs; negative fixture; unsupported operations; independent verifier
T11-T15	Predicate parity; superposition behavior; ancilla cleanup; endianness; no lookup-answer leakage
T16-T20	Disabled parity; shadow non-mutation; immediate-parent return; nested prerequisites; shared prerequisite

IDs	Required coverage
T21-T25	Cycle/depth refusal; rollback; restart without QPU; viewer read-only; protected-answer isolation
T26-T30	Zero-budget refusal; no paid fallback; atomic in-flight reservation; duplicate submit recovery; stale quote refusal
T31-T35	Timeout versus cancellation; failed/partial jobs; calibration/circuit provenance; simulator/hardware labeling; no-sample versus unsatisfiability
T36-T40	Exact test count reconciliation; external-assistance attribution; resource preflight; preserved active training; independent acceptance

Every gate report must give test IDs actually executed, exact command, source/environment identity, pass/fail/error/skip status, raw result location, and independent review status. "Collected" is not "executed". [P1]

20. Cloud decision record

Preferred progression

Route 1: minimum new spend. Use an existing authorized CPU host and local simulator, then IBM Open Plan. The standard Open allowance is 10 minutes of QPU use in a rolling 28-day window, not continuous server time. An eligible non-trial account that has used 20 minutes within a year may opt in to the documented one-time 180-minute allocation for the next year; check the account before budgeting it. [S3, S6]

Route 2: predictable paid hosting. Candidate Linux/public-IPv4 Lightsail bundles are 2 GB at $12/month, 4 GB at $24/month, and 8 GB at $44/month. Start by sizing the worker/control service, not by assuming these are adequate for the full HELM model or training. [S7]

Route 3: almost-zero cloud bill, more setup risk. Oracle's currently retrieved Always Free documentation lists Arm A1 resources equivalent to 2 OCPUs and 12 GB, subject to limits, capacity, and idle-instance reclamation. Only use this path after an Arm64 compatibility test and an external backup plan. Do not budget the older 4-OCPU/24-GB figures without account-specific verification. [S9]

Route 4: managed hybrid batches. Braket Hybrid Jobs can execute a containerized classical algorithm that calls a QPU and releases compute afterward. Use it for bounded experiment batches, not as the persistent HELM website or an implied free GPU server. CPU/GPU compute and QPU usage have separate charges. [S1, S2]

Quantum cost snapshot

Illustrative USD QPU-only charges for one task with 1,000 shots, calculated from the retrieved public Braket rates: Rigetti Cepheus **$0.725**; IQM Garnet **$1.75**; IonQ Forte **$80.30**. A shot is one circuit repetition, not one HELM reasoning episode. Ten Cepheus tasks of 1,000 shots each calculate to **$7.25**, before other services. [S1]

These are not quotes from the user's account. Device availability, billing eligibility, minimum shots, compilation feasibility, taxes, storage, host costs, and optional mitigation can change the actual bill. Reprice at submission. IQM and Rigetti are gate-based alternatives; hardware choice also depends on depth, connectivity, and errors. [S4]

IBM paid access publicly starts at **$96 per QPU minute**, billed per second, while Open is free within its rules. Azure Quantum has provider-specific prices and some free simulators; its current price page did not establish an automatic $500 quantum-credit entitlement, so that old claim is not used in this budget. D-Wave advertises a trial application, but no guaranteed recurring free allocation or fixed ongoing price was established in this research. [S10, S16, S8]

Proposed budget, not spending authorization

An example small pilot is $24/month for a 4 GB host plus a separately approved $10 QPU allowance, with storage/taxes/other services budgeted separately. The actual default in `config/policy.json` is **zero cloud spend and no submission permission**. Existing HELM training allowances are not automatically available to this new project.

For Braket, avoid IPv6-only Lightsail for the first integration: the SDK's current regional/device documentation states it does not support IPv6-only networks. [S4]

AWS's pricing page describes a free local simulator and a conditional managed-simulator allowance of one hour per month for the first twelve months of Braket use. Do not assume eligibility for an existing account or confuse a simulator allowance with free QPU access. [S1, S11]

21. Cloud runbook

First profile and reproduce the admitted HELM command on a compatible Linux container in an authorized environment. Resolve storage, ports, health checks, checkpoint loading, and restart behavior before selecting a VM. Separate persistent service needs from episodic training or experiment compute.

For IBM, create or select an Open instance in the supported region using the owner's account flow. Store credentials outside source control. Pin the instance, plan, and approved device policy. Stop when allowance is exhausted; never switch automatically to a paid instance.

For Braket, discover device properties and review prices in its region. Prepare a private result bucket and least-privilege execution identity. Record allowed devices, quotas, and the existing job inventory. Activate provider-side spending protection before pilot submissions; restrict the worker from changing that protection. The source documentation provides `create-spending-limit` and related commands, but this package deliberately does not execute them. [S5]

Submit one reviewed job; persist and reconcile its provider ID; decode and verify; inspect the actual cost record. Only then admit another bounded batch. Check that paid notebooks, instances, storage, addresses, and experiment resources have their intended lifecycle. Turning off the HELM process alone is not a cloud cleanup plan.

For a managed Braket Hybrid Job, package the admitted classical entry point in a compatible custom image and separately configure instance type, runtime limits, output storage, device, and accounting. Do not replace the current active HELM training job with this experiment. [S2, P1]

22. Required final delivery from the implementation team

Return the accepted ownership/communication map; exact source and dependency identities; baseline and host profile; typed contracts; oracle and verifier; classical/simulator/hardware reports separately; complete test matrix; successful or blocked RPM/RPA integration; retained-procedure reload evidence; actual cloud/cost state; unresolved findings; and the next eligible gate.

The evidence statement must distinguish code written, tests executed, hardware executed, source reviewed, and performance established. Preserve negative results. `RETAINED_RESEARCH_ONLY` is not `PROMOTED`.

Definition of done for the first milestone: one independently verified quantum-assisted procedure-construction episode with bounded cost, exact parent resumption, fresh-process reuse, and no interference with the admitted HELM baseline.

Sources

Provider facts were checked on September 27, 2026. URLs are operational references; availability and account eligibility must be rechecked before submission. The engineering prescriptions and proposed fixture are original design work, not claims that these providers validated CAS.

- **[P1]** Private project coordination note, `OWNER_CORRECTION_ROUND.txt`, branch `codex/helm-three-lead-delivery-20260927`, blob `dfd0b65104b88052419213627620a926d9838d42`. https://github.com/LittleHenderson/noetic-compass/blob/codex/helm-three-lead-delivery-20260927/OWNER_CORRECTION_ROUND.txt
- **[S1]** AWS, Amazon Braket pricing. https://aws.amazon.com/braket/pricing/
- **[S2]** AWS, Working with Amazon Braket Hybrid Jobs. https://docs.aws.amazon.com/braket/latest/developerguide/braket-jobs.html
- **[S3]** IBM Quantum, Plans overview. https://quantum.cloud.ibm.com/docs/en/guides/plans-overview
- **[S4]** AWS, Supported regions and devices. https://docs.aws.amazon.com/braket/latest/developerguide/braket-devices.html
- **[S5]** AWS, Cost tracking and saving, including QPU spending limits. https://docs.aws.amazon.com/braket/latest/developerguide/braket-pricing.html
- **[S6]** IBM, Doubling down on open-access quantum computing, March 16, 2026. https://www.ibm.com/quantum/blog/open-plan-updates
- **[S7]** AWS, Lightsail pricing. https://aws.amazon.com/lightsail/pricing/
- **[S8]** D-Wave, Leap quantum cloud service and trial route. https://www.dwavequantum.com/solutions-and-products/cloud-platform/
- **[S9]** Oracle Cloud, Always Free resources. https://docs.oracle.com/en-us/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm
- **[S10]** IBM Quantum, Products and access plans. https://www.ibm.com/quantum/products
- **[S11]** AWS, Submitting quantum tasks to simulators. https://docs.aws.amazon.com/braket/latest/developerguide/braket-submit-tasks-simulators.html
- **[S12]** IBM Quantum, Execution modes. https://quantum.cloud.ibm.com/docs/en/guides/execution-modes
- **[S13]** Ashley Montanaro, Quantum walk speedup of backtracking algorithms, 2015. https://arxiv.org/abs/1509.02374
- **[S14]** Challenges and Opportunities in Quantum Optimization, research review. https://arxiv.org/html/2312.02279v3
- **[S15]** Ashley Montanaro, Quantum speedup of Monte Carlo methods, 2015. https://arxiv.org/abs/1504.06987
- **[S16]** Microsoft Learn, Pricing plans for Azure Quantum providers. https://learn.microsoft.com/en-us/azure/quantum/pricing